DATABASE MANAGEMENT SYSTEMS · ASSIGNMENT 4

Relational Database vs. NoSQL Database

A comparative report on scalability, consistency, performance and use cases

Project	FoodExpress – Online Food Delivery System	Submitted by	Dhruvan Karthik
DBMS	MySQL Community Server 8.0.46	GitHub	github.com/hidhruvan-prog
Database	`foodexpress`	Date	25 October 2026

1. Introduction

Choosing a database is one of the longest-lasting decisions in a software system. For decades the **relational database management system (RDBMS)** – MySQL, PostgreSQL, Oracle, SQL Server – was the default for almost every application. From the late 2000s, web-scale companies faced data volumes, traffic levels and data shapes that a single relational server handled poorly, which led to a family of non-relational systems collectively called **NoSQL** ("Not only SQL"). This report compares the two approaches on scalability, consistency, performance and use cases, explains why organizations pick different technologies, and applies the comparison to the FoodExpress food-delivery system developed in this course.

2. Overview of the Two Models

2.1 Relational databases

- Data is stored in **tables** (relations) of rows and columns with a **fixed schema** declared in advance with DDL.
- Relationships are expressed with **primary and foreign keys** and combined at query time with **joins**.
- A declarative, standardized language – **SQL** – is used for definition, querying and manipulation.
- Transactions follow **ACID**: Atomicity (all or nothing), Consistency (constraints always hold), Isolation (concurrent transactions do not see each other's partial work) and Durability (committed data survives crashes).
- Normalization (Assignment 3) removes redundancy so that each fact is stored once.

2.2 NoSQL databases

NoSQL is not one model but four main families, each optimized for a particular access pattern:

Family	Data model	Typical systems	Good at
Key–value	A distributed hash map: key → opaque value	Redis, Amazon DynamoDB, Riak	Caching, sessions, shopping carts, counters, leaderboards
Document	Self-describing JSON/BSON documents that can nest arrays and sub-documents	MongoDB, Couchbase, Amazon DocumentDB, Firestore	Catalogues, content, user profiles, data whose fields vary
Wide-column	Rows partitioned by key; each row holds many columns sorted on disk	Apache Cassandra, ScyllaDB, HBase, Google Bigtable	Very high write throughput, time series, event logs, messaging
Graph	Nodes and edges with properties	Neo4j, Amazon Neptune, JanusGraph	Highly connected data: social networks, recommendations, fraud rings

Common characteristics are a **flexible or schema-less** structure, **horizontal scaling** by partitioning (sharding) data across many commodity servers, built-in **replication**, and data modelled around **how it will be read** (often denormalized) rather than around entities.

2.3 One FoodExpress order in both models

The same order is shown below as relational rows (finalized design from Assignment 3) and as a single document, the way a document database such as MongoDB would usually store it.

Relational representation

```
-- Relational: spread across 4 tables, joined at read time
orders      (order_id=17, customer_id=4, restaurant_id=1,
             address_id=7, agent_id=2, status='Delivered',
             delivery_fee=0.00)
order_items (17, item_id=1, quantity=2, price_at_order=289.00)
order_items (17, item_id=4, quantity=1, price_at_order=219.00)
payments    (order_id=17, amount=797.00, method='UPI',
             status='Completed')
reviews     (order_id=17, food_rating=5, agent_rating=4)
```

Document representation

```
// Document: one self-contained record, no joins
{ "_id": 17,
  "customer": { "id": 4, "name": "Priya Nair" },
  "restaurant": { "id": 1, "name": "Spice Route Biryani House" },
  "deliverTo": { "line": "12, 5th Cross", "pincode": "560011" },
  "items": [ { "name": "Chicken Dum Biryani", "qty": 2, "price": 289 },
             { "name": "Chicken 65",         "qty": 1, "price": 219 } ],
  "payment": { "amount": 797, "method": "UPI", "status": "Completed" },
  "review":  { "food": 5, "agent": 4 },
  "status": "Delivered" }
```

The document is fast to read in one operation and maps directly onto application objects, but the customer and restaurant names are now copies. Renaming the restaurant means updating every order document – the same update anomaly that normalization removed in Assignment 3. That trade-off is the core of the comparison.

3. Comparison

3.1 Scalability

RDBMS traditionally scale **vertically** – a bigger server with more CPU, RAM and faster disks – and scale reads horizontally with **read replicas**. Scaling writes across many servers means **sharding**, which a classic RDBMS does not do on its own: joins, foreign keys and transactions that cross shards become hard. Large users therefore add a sharding layer, for example **Vitess** for MySQL, which originated at YouTube. Newer "distributed SQL" systems (Google Spanner, CockroachDB, TiDB, YugabyteDB) build automatic sharding into the database while keeping SQL and transactions.

NoSQL systems were designed to scale **horizontally** from the start. Data is partitioned by a key (for example by `customer_id`) and spread over a cluster; capacity grows by adding nodes, and Cassandra and DynamoDB rebalance data automatically. Because a query is expected to touch one partition, throughput grows almost linearly with cluster size – as long as the data model avoids cross-partition queries.

3.2 Consistency

An RDBMS provides **strong consistency** and ACID transactions over many rows and tables: after a commit, every reader sees the new value, and constraints such as foreign keys and CHECKs are always enforced. Many NoSQL systems follow **BASE** – *Basically Available, Soft state, Eventually consistent*: replicas may briefly disagree, and converge later.

The **CAP theorem** explains why: when a network partition splits a distributed system, it must choose between **Consistency** (refuse some requests) and **Availability** (answer with possibly stale data). Cassandra and DynamoDB favour availability by default but offer **tunable consistency** (for example, Cassandra's QUORUM

reads and writes, or DynamoDB's strongly consistent reads). The line has also blurred: MongoDB has supported multi-document ACID transactions since version 4.0, and DynamoDB offers transactional APIs. Even so, multi-entity integrity rules ("a payment must reference an existing order") are still enforced by the database only in an RDBMS; in NoSQL they become the application's responsibility.

3.3 Performance

- **Reads of a whole aggregate** (one order with its items) are faster in a document or key–value store: one lookup by key, no joins. An RDBMS must join several tables, which is fast with indexes but costs more as the number of tables and rows grows.
- **Writes at very high volume** (GPS pings, clicks, chat messages) favour wide-column stores such as Cassandra, whose log-structured merge-tree storage turns writes into sequential appends.
- **Ad-hoc and analytical queries** – "revenue by city and cuisine last month" – favour an RDBMS: SQL can join, group and filter any combination of columns, and the optimizer chooses the plan. In a NoSQL store, a query that does not match the partition key may need a full scan or a separately maintained index or table.
- **Latency at the extreme**: in-memory key–value stores such as Redis answer in well under a millisecond, which is why they are placed in front of relational databases as caches.

3.4 Use cases

Relational databases fit best when ...	NoSQL databases fit best when ...
Data is structured and relationships matter (orders, customers, accounts)	Data is semi-structured or its shape changes often (catalogues, content, IoT payloads)
Correctness of every transaction is critical (payments, banking, inventory, bookings)	Availability and low latency matter more than instant consistency (feeds, likes, view counts)
Queries are varied and not all known in advance (reporting, analytics, admin tools)	Access patterns are known and simple (get/put by key, latest N events for a user)
Data volume fits on one large server or a modest cluster	Write volume or data size needs many servers
Examples: ERP, core banking, e-commerce orders, student records, hospital systems	Examples: session stores, real-time location, chat history, product catalogues, recommendations

3.5 Summary table

Criterion	Relational (RDBMS)	NoSQL
Data model	Tables, rows, columns; fixed schema	Key–value, document, wide-column or graph; flexible schema
Schema changes	ALTER TABLE; migration needed	New fields can be added per record
Query language	Standard SQL with joins, subqueries, aggregates	Product-specific APIs/languages (MQL, CQL, Cypher); limited joins
Scalability	Vertical; read replicas; sharding via add-ons or distributed SQL	Horizontal by design; automatic partitioning
Consistency	Strong; ACID transactions across tables	Often eventual (BASE); tunable; transactions vary by product
Integrity	Enforced by DB: PK, FK, UNIQUE, CHECK	Mostly enforced by application code
Redundancy	Minimized by normalization	Accepted (denormalized) to speed up reads
Performance strength	Complex queries, joins, reporting	Simple high-volume reads/writes by key
Maturity & skills	Decades of tooling; SQL widely known	Newer; each product has its own model
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Cassandra, Redis, DynamoDB, Neo4j

4. Why Organizations Choose Different Database Technologies

No database is best on every axis, so organizations choose based on their requirements:

1. **Nature of the data** – tabular and related data suits an RDBMS; nested, variable or highly connected data suits a document or graph store.
2. **Consistency requirements** – money, stock levels and seat bookings must never be double-counted, so they need ACID guarantees. A “like” counter that is a second out of date harms no one.
3. **Scale and traffic pattern** – millions of writes per second or petabytes of data push towards horizontally scalable NoSQL or distributed SQL; most business applications fit comfortably in a single RDBMS.
4. **Query patterns** – unpredictable, analytical queries need SQL; fixed key-based lookups do not.
5. **Speed of development** – a flexible schema lets start-ups change data structures quickly; a strict schema catches errors early and documents the data.
6. **Cost, skills and ecosystem** – licences, managed cloud services, existing staff skills, reporting tools and regulations (auditability, data residency) all influence the choice.

As a result, large companies rarely use only one database. They practise **polyglot persistence**: each part of the system uses the store that fits its access pattern, and the systems are connected through events or change-data-capture pipelines.

5. Real-World Examples

The examples below are drawn from the companies' own public engineering write-ups and well-known industry case studies; they describe the general architecture, not every system each company runs.

Organization	Technology	Why it was chosen
Banks and payment networks	Relational (Oracle, IBM Db2, PostgreSQL)	Transfers must be atomic and auditable; strict ACID transactions and constraints are non-negotiable.
Amazon	Dynamo → Amazon DynamoDB (key–value)	Amazon's 2007 Dynamo paper describes a highly available key–value store built so that the shopping cart keeps accepting writes even during failures; this design led to the DynamoDB cloud service.
Netflix	Apache Cassandra (wide-column) alongside relational stores	Cassandra's multi-region replication and write throughput suit large volumes of data such as viewing history; transactional data such as billing is kept in a relational database.
Discord	MongoDB → Cassandra → ScyllaDB	Billions of chat messages outgrew a single MongoDB replica set; Discord moved messages to Cassandra (2017) and later to the Cassandra-compatible ScyllaDB (2023) for lower, more predictable latency.
Uber	MySQL with its own “Schemaless” layer	Uber kept MySQL as the reliable storage engine but built a sharded, append-only layer on top to get horizontal scale – a hybrid of relational storage and NoSQL-style access.
Facebook (Meta)	MySQL + TAO graph cache	The social graph is stored durably in sharded MySQL, while TAO serves the huge read volume of objects and associations from cache.
YouTube	MySQL + Vitess	Vitess was created to shard MySQL horizontally while applications keep using SQL.
Fraud detection teams	Graph databases (e.g. Neo4j)	Fraud rings show up as shared phones, devices and addresses between accounts – a pattern of connections that is natural to query as a graph and expensive as many SQL self-joins.

6. Application to FoodExpress

FoodExpress illustrates why one platform can sensibly use both families. Its core business data – customers, restaurants, menus, orders, order items, payments and reviews – is structured and related, and must be correct: a

payment must belong to a real order, and an order's items must come from the right restaurant. That is why the system of record is **MySQL**, with the constraints implemented in Assignment 6. Other workloads have different needs:

Workload	Requirement	Suitable store
Orders, payments, menus, customers	ACID, integrity, reporting	**MySQL** (relational) – this project
Live rider location (a GPS ping every few seconds)	Very high write rate, latest value only, geo-search	Redis (in-memory, geospatial commands); Cassandra for history
Login sessions, cart before checkout, OTPs	Sub-millisecond reads, automatic expiry (TTL)	Redis (key–value)
Restaurant search and menu browsing	Full-text search, flexible attributes	Elasticsearch or MongoDB (document), fed from MySQL
"Customers who ordered this also liked …"	Relationship traversal	Graph store, or offline analytics
Monthly business reports	Complex joins and aggregates	MySQL / data warehouse (Assignments 8–9)

> For a course-scale system, one MySQL database covers every requirement comfortably. NoSQL components would only be added when a specific workload – usually real-time location or caching – exceeds what the relational store can serve, and never at the cost of the transactional integrity of orders and payments.

7. Conclusion

Relational databases offer a fixed schema, powerful SQL, joins, and ACID guarantees enforced by the database itself, which makes them the right choice for structured, interrelated and business-critical data. NoSQL databases give up some of these guarantees for horizontal scalability, flexible schemas and very fast key-based access, which suits large-scale, high-velocity or loosely structured data. The two approaches are converging – NoSQL products add transactions while distributed SQL databases add horizontal scale – and most organizations combine them. The right choice depends on the shape of the data, the consistency it needs, the scale it must reach and the queries it must answer.